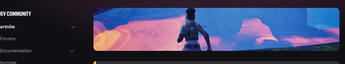
Join our community, grow your knowledge and learn from others!

Sign In

Don't have an Epic Games account? Sign up

Triggerable Interface and Scene Events

Create a common interface for triggerable puzzle components to use and scene events to signal them.



> Learn to use this **Beta** feature, but use caution when shipping with it.

This project uses multiple different Scene Graph components that are triggered by an action. This includes actions that are triggered by an event:

- Transform & mesh
- Toggle the visibility of a mesh
- Turn lights on or off

For example, all of the sphere lights in the image above are Scene Graph entities that have a component named `triggerable_light_component`, which is a component class that implements the `triggerable` interface. The different combinations of entities are triggered when the player steps on different white pedestals.

You can use Verse **interfaces** to define common functionality between unrelated objects or actions. In this case, the common functionality is triggering an action and the unrelated objects or actions are mesh visibility, mesh transformation, and turning lights on or off.

For more information about the Verse language features used on this page, see:

- Interface in Verse
- Scene Events

Define Triggerable Interface

You'll begin by creating a new empty Verse file and naming it `Triggerable.verse`. You'll define the `triggerable` interface. You can define default functionality and field values with interfaces, or leave them to child classes to define and implement. Then you'll define an event that is used to signal whether a Scene Graph entity that has a component that inherits from the triggerable interface has been triggered.

1. This file uses the `@editable` attribute on various interface fields. To do this, include the /Verse.org/Simulation module:

   Verse
   ```
   using { /Verse.org/Simulation }
   ```
   Copy full snippet (1 line long)

2. Interfaces in Verse are adjectives since they describe an action that child classes implement. Create a new interface named `triggerable`:

   Verse
   ```
   triggerable<public> := interface:
   ```
   Copy full snippet (1 line long)

3. You might want to keep track of whether or not a triggerable object has been triggered — for example, whether a light is on or off or whether a mesh is visible or invisible. It is possible that you might only want to trigger an action once. Add the following logic fields to `triggerable` to keep track of these:

   Verse
   ```
   var<protected> Triggered<public>:logic
   
   @editable
   OnlyTriggerOnce<protected>:logic = false

   var<protected> HasBeenTriggered<protected>:logic = false
   ```
   Copy full snippet (6 lines long)

4. Triggerable objects are used as pieces in the puzzles built later in this tutorial. Additionally, you might want to use triggerable objects as **decoy** pieces in a puzzle that are triggered by an action and the pieces act like they are part of the puzzle, but their triggered state does not contribute to or take away from a puzzle being solved. Add a `PuzzlePiece` logic field to keep track of whether or not this triggerable object is a puzzle piece.

   Verse
   ```
   @editable
   PuzzlePiece:logic = false
   ```
   Copy full snippet (2 lines long)

5. For even more flexibility, you might also want a puzzle where some lights must be on and some must be off for the puzzle to be considered solved. Add a `SolvedStateTriggered` logic field to indicate whether this triggerable object puzzle piece is considered solved if it is triggered or not triggered.

   Verse
   ```
   @editable
   SolvedStateTriggered:logic = true
   ```
   Copy full snippet (2 lines long)

6. Declare three functions: `PerformAction`, `PerformReverseAction`, and `PostTrigger`. Since `triggerable` is an interface and the interface does not provide a base implementation for these functions, any child class of `triggerable` must provide its own implementation of each of these functions.

   Verse
   ```
   # Action performed when transitioning from not triggered to triggered state
   PerformAction():void

   # Action performed when transitioning from triggered to not triggered state
   PerformReverseAction():void

   # Action performed after PerformAction or PerformReverseAction is called
   PostTrigger():void
   ```
   Copy full snippet (9 lines long)

7. Define three additional functions: `Trigger`, `IsSolvedState`, and `SetInitialTriggeredState`. These three functions all have a base implementation provided in the `Triggerable` interface. `Trigger` triggers the action for the component that implements this interface, `IsSolvedState` decides whether the component is in a solved triggered state, and `SetInitialTriggeredState` determines what the initial triggered state of a component is upon beginning of play.

   Verse
   ```
   # Trigger the appropriate action
   Trigger():void =
       if
           (OnlyTriggerOnce? and not HasBeenTriggeredOnce?) or (not OnlyTrigg
           then:
               if (Triggered?):
                   PerformReverseAction()
               else:
                   PerformAction()
               PostTrigger()
               set HasBeenTriggeredOnce = true

   # Is this triggerable in a solved state
   IsSolvedState<localizes><transact>:void =
       Triggered = SolvedStateTriggered

   # Set the initial triggered state of this object - default implementation
   SetInitialTriggeredState():void =
       set Triggered = false
   ```
   Collapse code Copy full snippet (70 lines long)

Define Triggered Event

This puzzle project also uses Scene events to send events up or down the Scene Graph hierarchy to trigger actions, like transforming a mesh or toggling a light on or off. In this case, we define an event named `triggered_event`. This event is sent up the Scene Graph hierarchy when a triggerable object is triggered. This event is handled by a Scene Graph component to determine whether or not a puzzle is solved.

1. Scene events require the Scene Graph API. To use the Scene Graph API, include the /Verse.org/SceneGraph module:

   Verse
   ```
   using { /Verse.org/SceneGraph }
   ```
   Copy full snippet (1 line long)

2. Add a Scene event named triggered_event:

   Verse
   ```
   triggered_event<public> := class(scene_event):
   ```
   Copy full snippet (1 line long)

3. Add two fields to this event, the first, an optional indicating the entity that is triggered, and the second, a logic indicating whether or not this entity has been triggered:

   Verse
   ```
   TriggeredEntity:?entity = false

   Triggered:logic = false
   ```
   Copy full snippet (3 lines long)

Next Steps

Next, you'll create the Puzzle Component to manage a puzzle and determine when it is solved.

Final Code for Triggerable.verse

Verse
```
<#
    Interface for triggering an action on a component.
    Any component that you want to be triggered when the touch_component has it's
    must implement this interface.
#>
using { /Verse.org/Simulation }

OnlyTriggerOnceToolTip<public><localizes><message>"Whether this triggerable compon
SolvedStateTriggeredToolTip<public><localizes><message>"Whether this puzzle pieces
PuzzlePieceToolTip<public><localizes><message>"Whether this entity is part of a pu

triggerable<public> := interface:

    @editable:
        ToolTip := OnlyTriggerOnceToolTip
    OnlyTriggerOnce<protected>:logic = false

    @editable:
        ToolTip := SolvedStateTriggeredToolTip
    SolvedStateTriggered:logic = false

    # Is this a puzzle piece?
    @editable:
        ToolTip := PuzzlePieceToolTip
    PuzzlePiece:logic = false

    # Is this currently triggered?
    var Triggered<public>:logic = true

    var<protected> HasBeenTriggeredOnce<protected>:logic = false

    # Move when triggered
    Trigger():void =
        if
            (OnlyTriggerOnce? and not HasBeenTriggeredOnce?) or (not OnlyTriggerOn
        then:
            if (Triggered?):
                PerformReverseAction()
            else:
                PerformAction()
            PostTrigger()
            set HasBeenTriggeredOnce = true

    # Action performed when not in triggered state
    PerformAction():void

    # Action performed when already in triggered state
    PerformReverseAction():void

    # Action performed after PerformAction or PerformReverseAction is called
    PostTrigger():void

    # Is this puzzle piece solved?
    IsSolvedState<localizes><transact>:void=
        Triggered = SolvedStateTriggered

    SetInitialTriggeredState():void =
        set Triggered = false

<#
    Scene event for triggering events to be sent through the scene graph when a tr
#>
using { /Verse.org/SceneGraph }
triggered_event<public> := class(scene_event):

    # Whether or not trigger occurred
    Triggered:logic = false

    # Triggering entity
    TriggeredEntity:?entity = false
```
Collapse code Copy full snippet (70 lines long)

Puzzle Component
Create a puzzle manager component to determine whether a puzzle is solved.

gameplay verse programming scene graph